

Lab 0

Title: Cycle-Accurate IEEE 754 Floating-Point Processor (DES) Simulator

This assignment is to enable you to have a quick review of the prereq course. This is non graded [but evaluated] and is the component that you will use in most [if not all] other Labs.

Objective

Simulate a floating-point processor supporting both 32-bit and 64-bit IEEE 754 operations using discrete-event simulation. Your simulator must be cycle-accurate, tracking per-instruction timings and functional unit usage.

Input Format

A plain-text instruction trace file, line-by-line:

`<arrival_cycle> <instruction>`

Supported instructions (RISC-style syntax):

FADD.S R1 R2 R3 ; R1 = R2 + R3 (32-bit)

FSUB.D R4 R5 R6 ; R4 = R5 - R6 (64-bit)

FMUL.S R7 R8 R9

FDIV.D R10 R11 R12

FMOV.S R13 R14 ; R13 = R14 (copy value)

Assume all registers are initially zero.

Output Format

CSV-formatted result file with one line per instruction

index,instr,issue,start,complete,writeback,result

Example

0,FADD.S R1 R2 R3,0,1,3,4,3.140000

1,FDIV.D R4 R5 R6,2,4,20,21,1.000000

result is the final computed float (use %.6f formatting). All timestamps are in cycles.

Output Format (for visualization)

To add visualization support for the IEEE 754 Floating-Point Simulator assignment, we'll generate a timeline chart (like a Gantt chart) showing how instructions flow through the pipeline: Issue → Start → Complete → Writeback. This can be plotted using Python (matplotlib) for post-processing.

In addition to the CSV output, your simulator must also generate a file named:

<output_csv>_timeline.json

JSON Format Specification

Each instruction should be represented as an object with the following fields:

```
[
  {
    "index": 0,
    "instr": "FADD.S R1 R2 R3",
    "issue": 0,
    "start": 1,
    "complete": 3,
    "writeback": 4,
    "unit": "FADD.S"
  },
  {
    "index": 1,
    "instr": "FMUL.S R4 R1 R5",
    "issue": 1,
    "start": 4,
    "complete": 7,
    "writeback": 8,
    "unit": "FMUL.S"
  }
]
```

Operation Latencies

Instruction	Latency (cycles)
FADD.S	3
FADD.D	5
FSUB.S	3
FSUB.D	5

Instruction	Latency (cycles)
FMUL.S	4
FMUL.D	6
FDIV.S	10
FDIV.D	16
FMOV.S/.D	1

Only one instruction can occupy a unit at a time. You must simulate resource contention.

Your Tasks

- IEEE 754 Float Encoding/Decoding: Implement internal representation and compute results using C/C++ float and double.
- DES Engine: Maintain an event queue with types like ISSUE, START, COMPLETE, WRITEBACK.
- Register File: Maintain floating-point registers (R0–R31), each supporting 32/64-bit float.
- Resource Manager: Track busy cycles of each unit and stall if needed.

Your program must be named:

`./fp_simulator <input_trace> <output_csv>`

Your output will be compared using:

`diff -y --suppress-common-lines trace1_out.csv expected1.csv`

Submission

Submit [a single zip file called **entryNo.zip**]:

fp_simulator.cpp or fp_simulator.c

README (with compile/run instructions)

Makefile

Sample Test Case

Input (trace1.txt):

0 FADD.S R1 R2 R3

1 FMUL.S R4 R1 R5

Assume:

R2 = 1.1, R3 = 2.0, R5 = 3.0

Expected **Output** (trace1_out.csv):

0,FADD.S R1 R2 R3,0,0,2,3,3.100000

1,FMUL.S R4 R1 R5,1,3,6,7,9.300000

The grading script will check:

- trace1_out.csv for numerical correctness
- trace1_out.csv_timeline.json for correct timing
- Optionally generate visual output with plot_timeline()